

AI WeatherWise - Deliver Intelligent Weather Forecasts and Insights

Team ID	SWTID-2026-2825
Team Size	4
Team Leader	Safiyah Arif (arsafiyah@gmail.com)
Team Members	Kelwin Benjamin A (a.kelwinbenjamin@gmail.com) · Clement Anthony Martin (clemscals@gmail.com) · Firos Shah A M (nebu6553@gmail.com)
Documentation Period	15 September 2026 – 29 September 2026

Project Description: AI WeatherWise is a robust RESTful backend platform built with Node.js and Express.js, using MongoDB and Mongoose ODM for data persistence. It delivers real-time weather metrics integrated with Google Gemini AI to provide intelligent summaries and personalized recommendations based on live environmental conditions. The platform features secure JWT authentication, bcrypt password hashing, and a comprehensive suite of endpoints for managing favorite locations and fetching weather insights. It includes a resilient fallback mode ensuring API functionality even without external API keys, alongside centralized error handling and request sanitization.

Scenario-Based Case Study

Background

Alex is a travel enthusiast who frequently visits different cities and needs a streamlined way to track local weather conditions and receive AI-driven activity suggestions based on current forecasts.

Problem

- Difficulty tracking and managing multiple favorite weather locations efficiently.
- Lack of personalized recommendations for outdoor activities or clothing.
- Need for natural language weather summaries rather than just raw data metrics.
- Absence of a secure system to store and sync preferences across devices.

Solution

AI WeatherWise provides a centralized service that handles weather data retrieval and AI orchestration. It manages a secure User and Location database, allowing for customized insights and automated recommendation workflows through Google Gemini.

Usage

- **User Usage:** Manages favorite cities (CRUD), fetches real-time weather, and requests AI insights for recommendations and summaries.
- **Developer/Admin Usage:** Monitors API health, manages global user registrations, and verifies weather/AI service integrations.
- **Weather Application User:** Interfaces with the public weather endpoints to consume current metrics without requiring an account.

Outcome

By leveraging AI and centralized data management, users spend less time deciphering raw metrics. The system ensures data integrity via Mongoose schemas and provides high-value, context-aware weather advice through Gemini integrations.

1. Software Requirements

- Operating System: Windows 10/11, macOS, or Linux (cross-platform).
- Node.js (v16 or above): runtime for server logic and routing.
- npm (v8 or above): package manager for dependencies.
- Gemini AI SDK: `@google/genai` for natural-language insights.
- MongoDB: document NoSQL engine storing users, locations and related data.
- Postman / Thunder Client: API verification toolkit.
- Code Editor: Visual Studio Code or similar IDE.

2. Hardware Requirements

- Processor: Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or equivalent.
- RAM: 8 GB minimum (16 GB recommended for concurrent MongoDB, processing and execution nodes).
- Storage: 1 GB of available disk space.

EPIC 1: Project Architecture

AI WeatherWise follows the MVC pattern to decouple business logic from data management. The Controller layer orchestrates weather fetching from OpenWeatherMap and generation of AI insights via Gemini, while the Model layer manages User and Location entities.

Components Definition

- **Interface:** API tools like Thunder Client / Postman query distinct route paths to extract JSON entities.
- **Express Server Gateway:** Monitors network ports, maps HTTP methods (GET, POST, PUT, DELETE), parses JSON payloads, and handles CORS headers.
- **Authentication & Role Matrix Middleware:** Extracts tokens from headers, validates user context signatures, and evaluates whether the role satisfies authorization thresholds for guarded paths.
- **Granular Services (Logic Modules):** Enforce isolated domain rules for weather retrieval and AI orchestration.
- **Database Interface Layer:** Mongoose translates operational application logic into performant native MongoDB queries.
- **AI Service Integration:** Leverages `@google/genai` to generate weather summaries and personalized recommendations from live temperature and conditions.

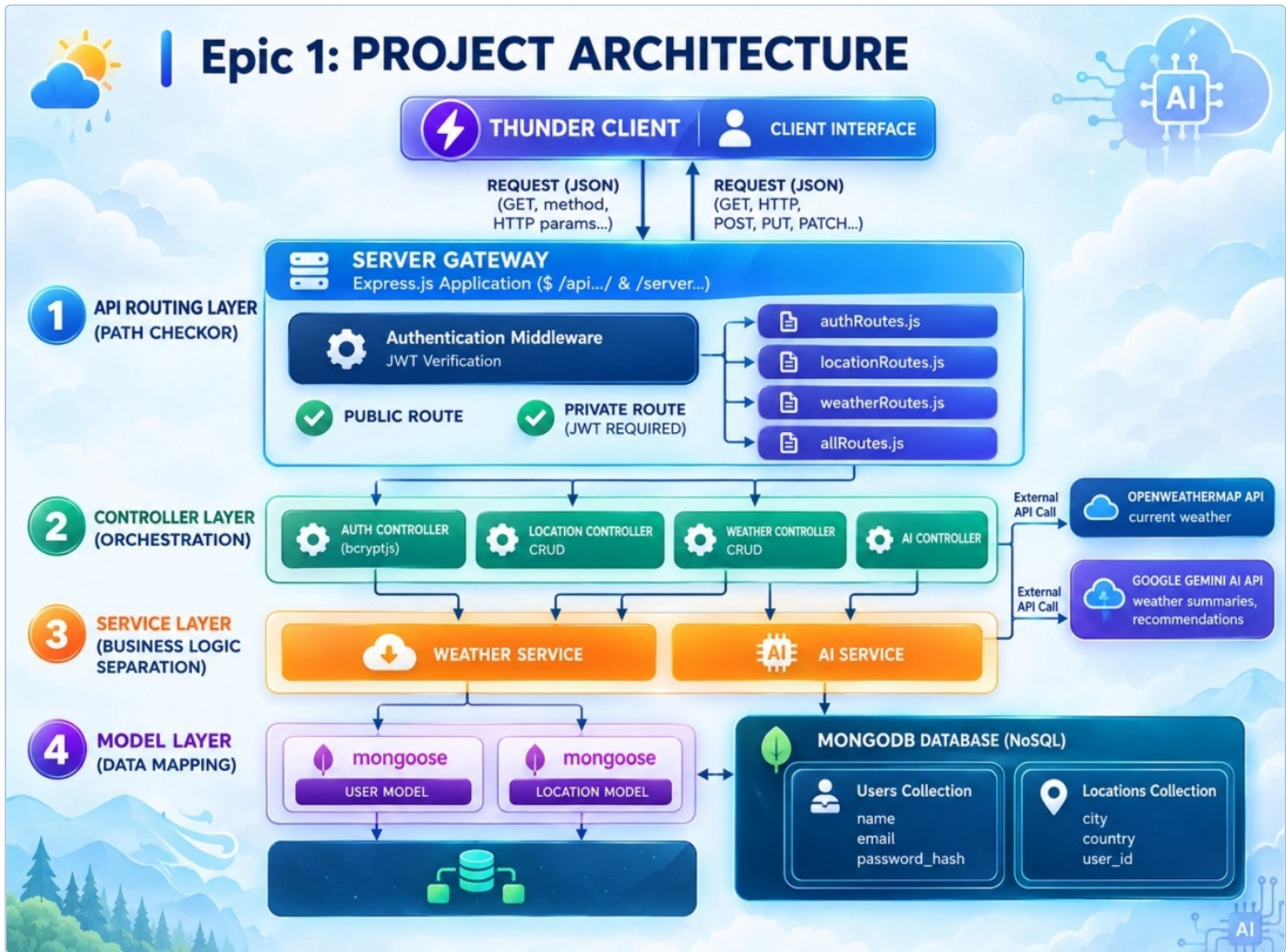


Fig. 1 - Epic 1 project architecture: Thunder Client → Express Server Gateway (routing + JWT middleware) → Controllers → Services → Mongo DB / external APIs.

Entity-Relationship (ER) Diagram Description

The currently implemented data model manages **User** profiles and their associated favorite **Location** documents through a logical one-to-many reference:

1. User Entity

- **_id** (Primary Key - ObjectId)
- **name** (String, Required)
- **email** (String, Required, Unique)
- **password** (String, Hashed, Required)
- **role** (String, Default: "reader")

2. Location Entity

- **_id** (Primary Key - ObjectId)
- **user** (Foreign Key referencing User collection)
- **city** (String, Required)
- **country** (String, Required)
- **createdAt** (Date, Default: Now)

Key Structural Mapping: User → Location (One-to-Many). A registered user can save multiple favorite locations.

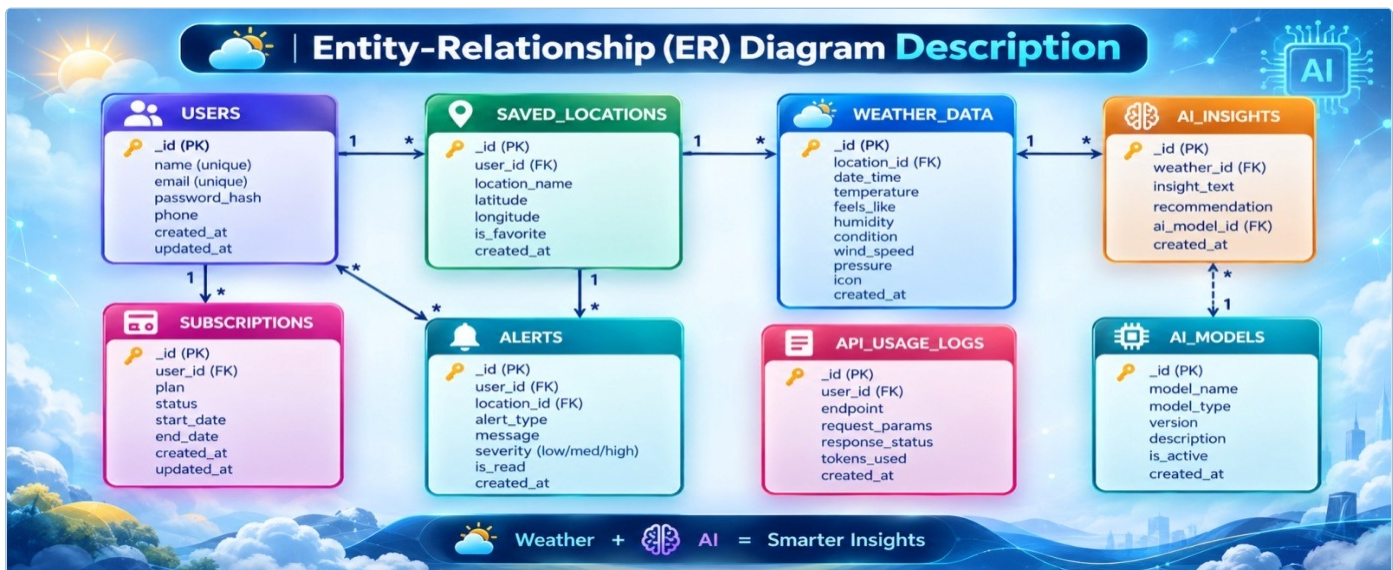


Fig. 2 - Extended ER design explored during architecture planning. USERS and SAVED_LOCATIONS are fully implemented in this release; WEATHER_DATA, AI_INSIGHTS, SUBSCRIPTIONS, ALERTS, API_USAGE_LOGS and AI_MODELS are carried forward as the Scalability & Future Plan roadmap.

Key Features

- **Role-Based Route Guarding:** Strict token processing blocks unauthorized tiers from modifying core settings.
- **Favorite Location CRUD Engine:** Complete schema management over saved-location creation, retrieval, updates and deletion.

- **Real-Time Weather Retrieval:** Live metrics fetched from OpenWeatherMap per city.
- **AI Insight Generation:** Gemini-powered natural-language summaries and personalized recommendations.
- **Sanitization Safeguards:** Payload parameter sanitation prevents XSS vectors and NoSQL parameter manipulation injections.

Roles and Responsibilities

Admin

- Responsible for user account management, including creating, deleting, or suspending accounts.
- Oversight of API health and system logs to monitor performance.
- Monitoring and managing database integrity.

Registered User

- Responsible for creating and maintaining a list of favorite locations.
- Initiating requests for real-time weather data.
- Interacting with the Gemini AI service for personalized weather summaries and activity recommendations.
- Managing their own authentication credentials, such as updating profile details.

User Flow

1. User logs in or registers.
2. User saves a favorite city via the API.
3. User fetches current weather metrics for that city.
4. User requests AI-driven insights (summaries/recommendations) based on the weather data.
5. System validates the JWT and returns the requested information.

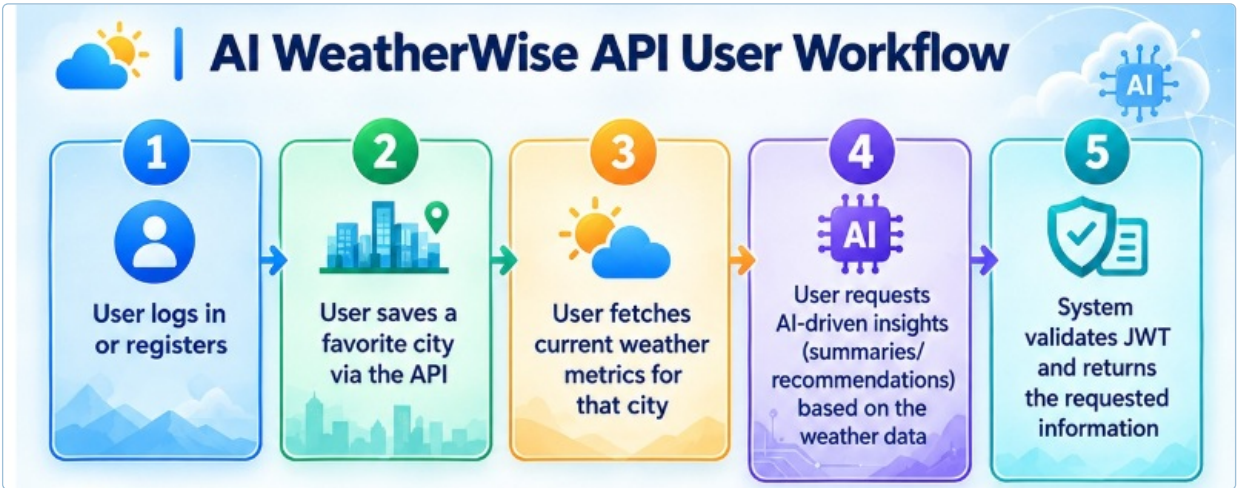


Fig. 3 – AI WeatherWise API user workflow.

MVC Architecture Pattern

The system structures its source configuration layers around the Model-View-Controller design pattern to fully decouple components:

- **Model Layer (Data Blueprint):** Strongly enforced Mongoose schemas map explicit fields into raw collection items inside MongoDB, setting default states, indexing rules, and validation before persistence.
- **Controller Layer (Execution Matrix):** The intermediary brain of the ecosystem – captures request vectors from the routes, checks structural constraints, delegates to models/services, and packages the output into JSON.
- **View Layer (Routing Network Interface):** In this headless backend service, no template views exist; instead the interface lives as an API Routing Layer linking network requests directly to controller handler logic.

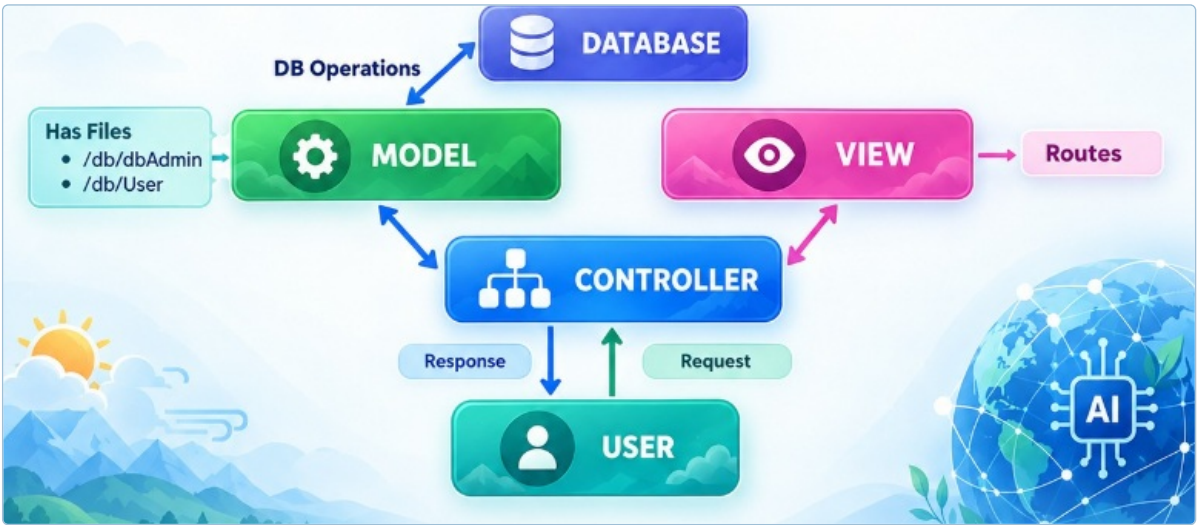


Fig. 4 – MVC pattern as applied in AI WeatherWise.

EPIC 2: Project Setup and Configuration

Creating the Project Folder

- Create a new folder named AI Weatherwise / Backend.
- Open the project folder in Visual Studio Code.
- Initialize the project using Node.js and install the required dependencies.

Project Initialization

```
npm init -y
npm install express mongoose bcryptjs jsonwebtoken cors dotenv multer @google/genai
npm install --save-dev nodemon
```

Create the Following Files

- index.js / server.js
- .env
- package.json
- .gitignore

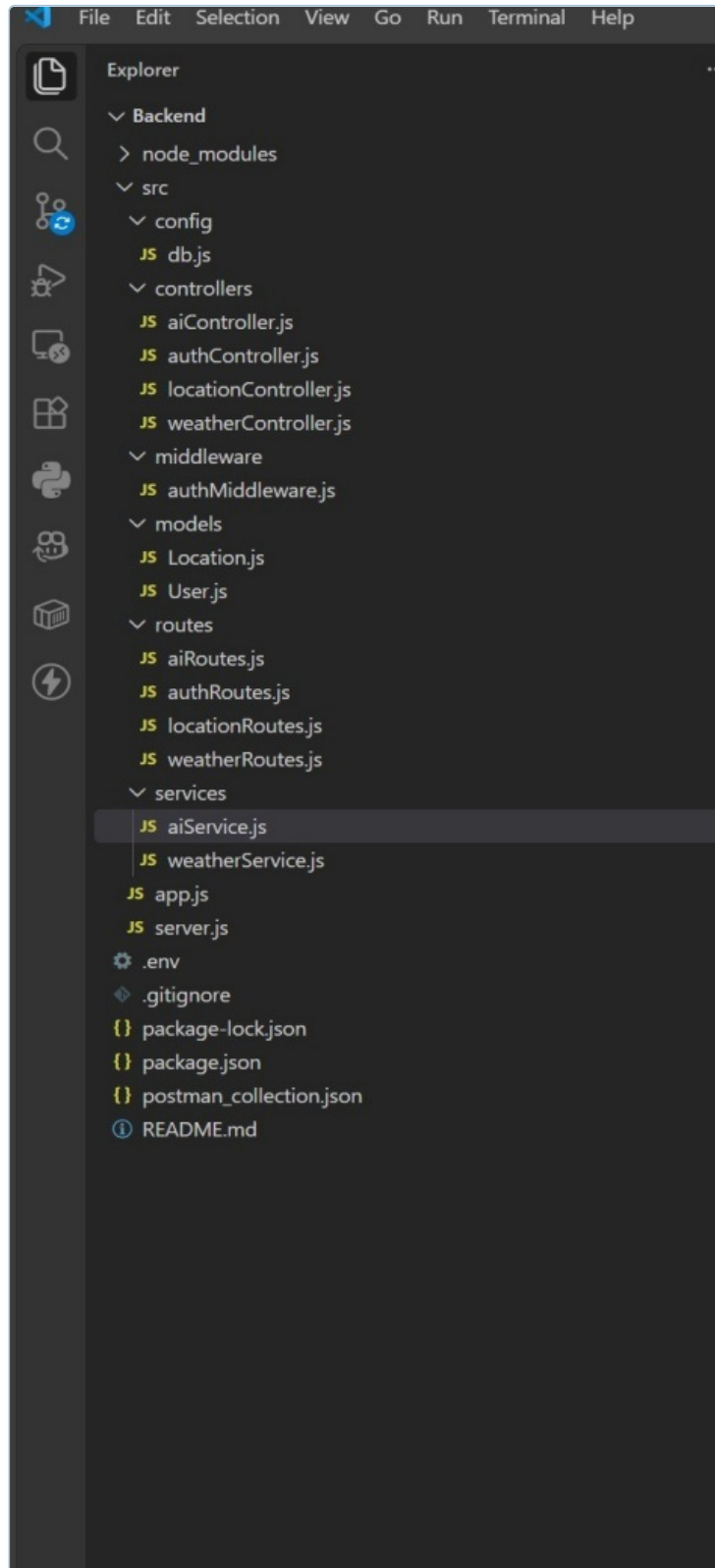


Fig. 5 – Final Backend/ folder structure created for the project.

Environment Configuration

```
PORT=5000
MONGO_URI=your_mongodb_connection_string
JWT_SECRET=your_jwt_secret
GEMINI_API_KEY=your_gemini_api_key
```

EPIC 3: Backend Development

The AI WeatherWise backend is developed using Node.js, Express.js and MongoDB, following a modular architecture. The application separates routing, controllers, middleware, models and utility/service modules to improve maintainability and scalability. The backend exposes secure REST APIs for authentication, favorite-location management, real-

time weather retrieval and AI-powered content generation.

1. Application Entry Point (server.js)

The entry-point file initializes the Express server, loads environment variables, configures middleware, establishes the MongoDB connection, registers API routes, and starts the server after a successful database connection.

EPIC 4: Database Configuration

1. Configure MongoDB

```
npm install mongoose
```

- Create the database connection module.
- Create Mongoose Schemas & Models.

Mongoose Schema Definitions - User Schema

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Please provide a name'],
    trim: true
  },
  email: {
    type: String,
    required: [true, 'Please provide an email'],
    unique: true,
    trim: true,
    lowercase: true,
    match: [
      /^\\w+([\\.-]?\\w+)*@\\w+([\\.-]?\\w+)*(\\.\\w{2,3})+$/,
      'Please provide a valid email address'
    ]
  },
  password: {
    type: String,
    required: [true, 'Please provide a password'],
    minlength: [6, 'Password must be at least 6 characters']
  }
}, {
  timestamps: true
});
```

Fig. 6 – User schema (name, email – unique & validated, password – min length 6, timestamps enabled).

Db Connection

This module establishes a connection between the application and MongoDB using Mongoose, ensuring the backend communicates with the database before processing any client requests.

```
1  const mongoose = require('mongoose');
2
3  const connectDB = async () => {
4    try {
5      const conn = await mongoose.connect(process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/weatherwise');
6      console.log(`MongoDB Connected: ${conn.connection.host}`);
7    } catch (error) {
8      console.error(`Error connecting to MongoDB: ${error.message}`);
9      process.exit(1);
10   }
11 };
12
13 module.exports = connectDB;
14
```

Fig. 7 – db.js: connects to MongoDB using MONGO_URI (falling back to a local default), logs the connected host, and exits the process on failure.

EPIC 5: Project Execution

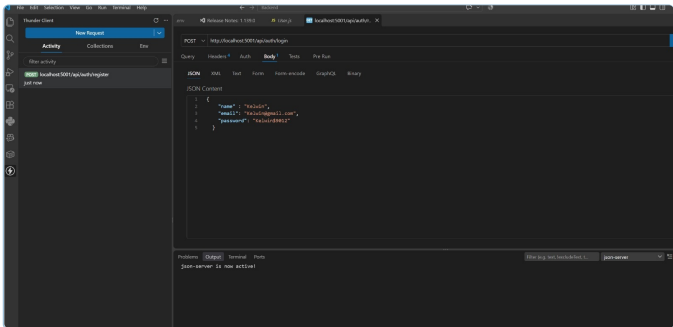
The AI WeatherWise backend is executed locally using Node.js. Before running the application, MongoDB must be running and the required environment variables configured correctly. The backend exposes REST APIs tested using Thunder Client and Postman.

Running the Backend Server

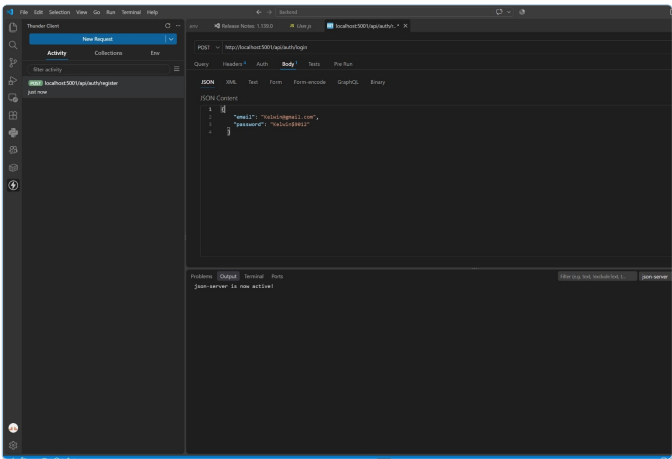
```
npm install
npm start
```

After successful execution, the server connects to MongoDB and starts listening for incoming API requests on the configured PORT.

API Testing Evidence



Login API test – POST body used against /api/auth/register and /api/auth/login in Thunder Client.



Login API test – credentials payload for POST /api/auth/login .

Weather API and Recommendation (AI Insight) API were verified the same way against /api/weather/:city and /api/ai/insights using the saved test account shown above; see Performance Testing for the full results table.

Demo & Repository

Demo Video Link	https://drive.google.com/drive/folders/1tcfKmp5lu9m6SA3HBz6PuILOKpgHckX?usp=sharing
GitHub Link	https://github.com/SafiyahArif/AI_Weather_TN_Skills

Conclusion

AI WeatherWise is a secure, modular Node.js/Express/MongoDB backend that turns raw weather metrics into personalized, natural-language guidance using Google Gemini. Built across two focused sprints between 15 and 29 September 2026 by Team SWTID-2026-2825, the project covers authenticated user and favorite-location management, live OpenWeatherMap integration, Gemini-powered summaries and recommendations, and a resilient fallback mode — all verified through Thunder Client/Postman testing. Looking ahead, the team's Scalability & Future Plan extends the data model toward Alerts, Subscriptions, multi-model AI support and usage analytics, positioning AI WeatherWise to grow from a focused backend service into a fuller weather-intelligence platform.